

Structured 3D from Language. Every Gaussian Has a Name.

Recursive semantic-to-geometric decomposition via Gaussian splatting. A language model breaks prompts into composition trees, each leaf becomes a named Gaussian primitive.

May 2026 / Pre-seed

The Problem

Current text-to-3D produces monolithic, non-divisible output

Today's generative approaches predict pixels or radiance fields directly through diffusion. They do not produce structured, decomposed, editable scenes. None use compositional Gaussians. None produce labeled parts. None allow editing without full re-generation.

- Video diffusion (Sora, Runway): non-divisible frame sequences
- Image diffusion (DALL-E, Midjourney): flat rasters, no geometry
- 3D diffusion (DreamFusion, Point-E): opaque radiance fields

Monolithic (Status Quo)

Input: prompt. Output: 10M floats.
No parts. No labels. Edit = start over.

Structured (SGS)

Input: prompt. Output: Gaussian space,
decomposed into named blobs. Edit one node.

Our Solution

Recursive Semantic-to-Geometric Decomposition (RSGD)

A language model decomposes prompts into composition trees. Each node is a named sub-concept with spatial relations. Terminal nodes become individual Gaussian splats. The tree IS the editable representation.

```
"a castle on a hill"
|
[Composer LM] --> Composition Tree: scene
|               +-- castle (pos=[0,0.8,0])
[Recursive      |   +-- tower_1 (cylinder+cone)
Renderer]       |   +-- keep (box+cone)
|               +-- hill (dome, green)
v
Editable Scene Graph + Real-time 3D Render
```

What Is Semantic Gaussian Splatting?

SGS unifies language understanding and 3D geometry under one equation. A single Gaussian primitive encodes both spatial structure and semantic meaning.

The SGS Primitive: $G = (\mu, \Sigma, \alpha, f)$

- μ (position): center point in splatting space
- Σ (covariance): shape and orientation of the ellipsoid
- α (opacity): transmittance weight during rendering
- f (features): semantic embedding vector encoding meaning

Key insight: alpha-compositing = strict superset of softmax attention.
Formally verified in Lean 4. Submitted to JMLR.

What Are Semantic Gaussian Blobs?

A blob is a pre-computed chunk of meaning stored as a Gaussian. One blob = one passage/concept/object compressed into a single primitive. Blobs bridge language memory and 3D scene composition.

Two-Pass Rendering

- Pass 1 (blobs): Retrieve background knowledge by cosine similarity
- Pass 2 (words): Alpha-composite current input over retrieved blob layer

This lets the system recall prior scenes, reuse memorized components, and compose new structures from existing blobs without retraining.

How We Use SGS + Blobs to Render

Four-stage pipeline from prompt to final scene

1. Decomposition: LM parses prompt into tree
2. Blob Mapping: nodes map to shape distributions
3. Gaussian Instantiation: terminals become $G=(\mu, \Sigma, \alpha, f)$
4. Alpha-Compositing: front-to-back transmittance rendering

```
PROMPT
|
[DECOMPOSER LM]
|
[BLOB MAPPING]
|
[INSTANTIATION]
|
[ALPHA-COMPOSE]
|
RENDERED 3D SCENE
```

Key Innovations

Five pillars from theory to implementation

Alpha-Compositing Superset Proof

Strict superset of softmax attention. Lean 4 verified. JMLR.

Recursive Decomposition

Prompts -> trees -> sub-trees -> Gaussians. Hierarchy = scene graph.

Conversation Memory

Each turn stored as blob. 40% recall at 90 turns. Flat cost.

Editable Scene Graphs (DSL v1)

Edit by name, reposition, recolor. Instant re-render.

OOV Policy

Unknown objects resolved via GloVe cosine NN (300 classes).

Demonstration: Model-Generated

Planck 1.3 (100M params, structure-only generation)



Limited by 512-token context. Hertz (1B+) produces richer trees.

Demonstration: Iteration 2 with Larger Model

Hand-authored composition trees rendered through the same pipeline. Target quality for Hertz-scale generation.



Applications

Structured, queryable, recomposable 3D scenes

- Game Asset Pipelines: editable scenes with labeled parts
- Architectural Visualization: descriptions to 3D walkthroughs
- Film Pre-visualization: script to scene layouts
- Education: labeled 3D models with queryable parts
- Conversational 3D: retrieve prior blobs, modify, re-render
- Accessibility: semantic labels for screen readers, spatial audio

Roadmap

From proof-of-concept to production API

Completed

- Planck 1.0-1.3: 100M LM, blob retrieval
- Raum 0.1: 300-class text-to-3D demo
- Raum 1.3: recursive decomposition pipeline
- JMLR: alpha-compositing proof
- Planck 1.4: conversation memory

Next

- Hertz 1.2 (Q3 2026): 1B param model
- Raum 2.0 (Q3 2026): Hertz-powered decomposer
- API Beta (Q4 2026): prompt to 3D JSON
- SDK (Q1 2027): Unity + Unreal plugins

Structured 3D. From Language. At Scale.

We are building the decomposition layer between language models and 3D rendering. Every Gaussian has a name, a parent, and a position in a tree.

Nikita Gorshkov, Founder

Pre-seed funding to scale Decomposer LM (1B), build API, hire rendering engineer.

github.com/feamando/sgs